



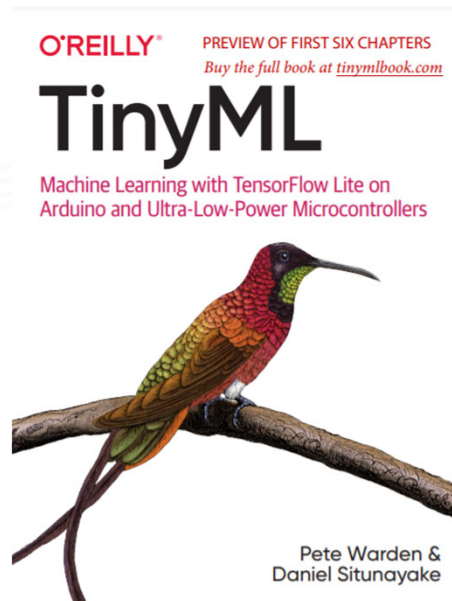
**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

SC4172 TinyML - Optimizing for TinyML Operations

Assoc Prof Nicholas Vun
College of Computing and
Data Science

References

Chapter 15, 16 of TinyML Book



Optimization Performance

Embedded systems don't have much computing power

- intensive calculations needed for neural networks can take longer than on most other platforms.

But embedded systems usually **operate** by sampling streams of sensor data **in real time**

- too **long processing time** might affect/restrict the sampling process
 - may **miss** one of the **event occurrences**
 - which can **reduce the accuracy**

Speeding up the model execution may also reduce the overall energy usage

- allow the device to run at a lower CPU frequency
- go to sleep in between inferences

Techniques to reduce the latency

Check the latency caused by the model

- if your neural network code is a small part of your overall system latency
 - speeding it up wouldn't make a big difference to the system's performance

A simple way to check this

- commenting out the call to `tflite::MicroInterpreter::Invoke()` in your application code
 - observe the difference it makes to the overall latency
- this function contains all of the inference calculations

Measuring the latency

In an ideal case

- you'll be able to calculate this change with a timer log statement or profiler

But even just blinking an LED and eye balling the frequency difference

- might be enough to give you a rough idea of what the speed increase is.

If the difference between running the model inference and not running is small

- not much to gain from optimizing the deep learning part of the code
- should focus on other parts of your application first.

Reducing the latency - Hardware

If you do need to speed up your neural network code

- are you able to use a more powerful hardware device?
- the easiest factor to change from a software perspective - worth considering

This may not be possible for many embedded products

- decision on which hardware platform to use is often made very early on or has been set externally

If you do have a choice, the biggest constraints are

- energy
- speed
- cost

Reducing the latency – Trade off

If possible, trade off energy or cost for speed by changing the chip you're using

- you might even get lucky in your search and discover a newer platform that gives you more speed
 - without losing either of the other main factors!

Latest microcontroller families for industrial applications

- Cortex-M
- RISC-V

Reducing the latency - Model

After switching hardware platforms

- the easiest place to have a big impact on neural network latency is at the architecture level.

It's usually possible to trade reduced accuracy for increased speed

- if you're able to start with as accurate a model as you can get at the beginning
 - you'll have a lot more scope for these trade-offs

So worthwhile to spend time on

- improving and expanding your training data throughout the development process

Reducing the latency - Model

Focus on model architectures

- better to change the model entirely if you can
 - rather than improving the speed of running it
- a lot easier to swap out machine learning models (than to switch algorithms of program)
 - because each model is just a functional 'black' box that takes in input data and returns numerical results

After you have a good set of data gathered

- comparatively easy to replace one model with another in the training scripts
- experiment with removing individual layers from a model
 - neural networks tend to degrade extremely gracefully
- observe their effect on accuracy and latency

Estimating Model Latency - MACs

Most neural network models spend the majority of their time running large matrix multiplications (MAC)

- every input value must be scaled by a different weight for each output value
- processing is approximately the number of input values times the number of output values for each layer in the network

Number of floating-point operations (FLOPs) for a fully connected layer

- equal to the size of the input vector, multiplied by the size of the output vector
- can figure out the work involved if you know those dimensions
 - approximated by using number of (FLOPs) that a network requires for a single inference run.
 - even though floating-point numbers are no longer involved (e.g. converted to 8-bit integer by quantization)

Estimating Model Latency - FLOPs

FLOPs are useful as a rough metric for how much time a network will take to execute

- all else being equal, a model that involves fewer calculations will run faster and in proportion to the difference in FLOPs
 - reasonable to expect a model that requires 100 million FLOPs to run twice as fast as a 200-million-FLOP version

Not entirely true in practice

- other factors like how well optimized the software is for particular layers that will affect the latency (e.g. using efficient vectorization)
- but a good starting point for evaluating different network architectures
- help establish what's realistic to expect for your hardware platform
 - 1-million-FLOP model take 100ms on a chip, expected that a different model requiring 10 million FLOPs will take about a second to calculate

Speeding up a Model

Model architecture design is still an active research field

- best starting point is to find some existing models that have been designed with efficiency in mind
- and then iteratively experiment with change

Many models have particular parameters that we can alter to affect the amount of computation required

- such as the input size expected.

Look at the FLOPs required for each layer

- try to remove particularly slow ones or substitute them with faster alternatives
 - E.g. using depthwise separable convolution instead of plain convolution

Quantization

Most programs that perform complex calculations (with million of operations) are very sensitive to numerical precision

- errors build up and give a result that's too inaccurate to use

Deep learning models are different

- able to cope with large losses in numerical precision during intermediate calculations
 - still produce end results that are accurate overall.
- seems to be a by-product of their training process
 - inputs are large and full of noise
- models learn to be robust to insignificant variations
 - focus on the important patterns that matter

Quantization

Hence model operating with 32-bit floating-point representations

- almost always more precise than is required for **Inference**.
- compare to Training which is a bit more demanding
 - because it requires many small changes to the weights to learn

Most inference applications that use just 8 bits to store weights and activation values

- can produce results that are indistinguishable from the floating-point equivalent
- especially suitable for embedded applications given that many such platforms have strong support for 8-bit MAC instructions that these models rely on
 - because such instructions are commonly used in signal-processing algorithms.

Post-Training Quantization

The easiest type of quantization to use is what's known as post-training weight quantization

- can shrink the model file size by 75% and offers some speed benefits
- weights are quantized down to 8 bits
- doesn't require any knowledge of the activation layer's ranges
 - but (still) need fast floating-point hardware

Post-training integer quantization

- means that a model can be executed without any floating-point calculation
- makes it the preferred approach for the use cases
- need to provide some example inputs during the model export process
 - to observe the ranges of the activation layer outputs

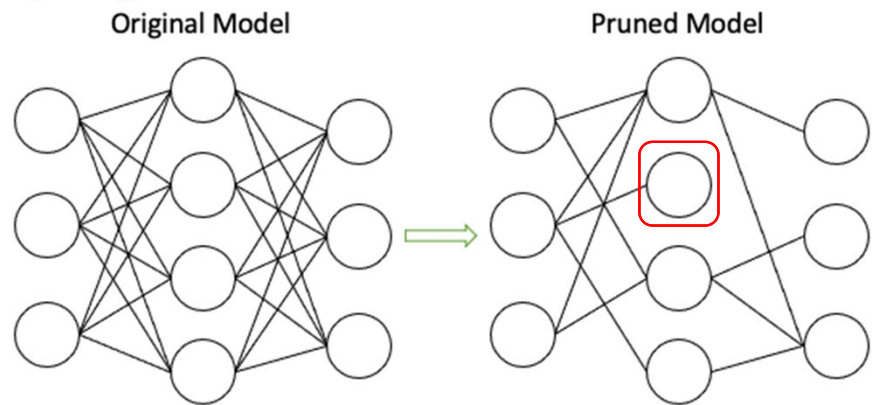
Weight Pruning

Pruning is another technique to reduce the model size after training

- a model compression technique (e.g. keras.prune_low_magnitude API)
- eliminates the weights with low magnitudes
 - not much contribution to the final model performance
- allows the model to be optimized for real-time inference for resource-constrained devices.

Both original and pruned model has the same architecture (?)

- weights with the low magnitude being set to zeros.
- pruned model is hence sparser
- $x_{pnu} \otimes m_j \otimes j_w t j x \otimes i z w s l \otimes s k j w j s h j$
 - $q f y j s h \sim \otimes r u w t \{ j r j s y$



Dropout

Dropout is a technique to help prevent overfitting during training.

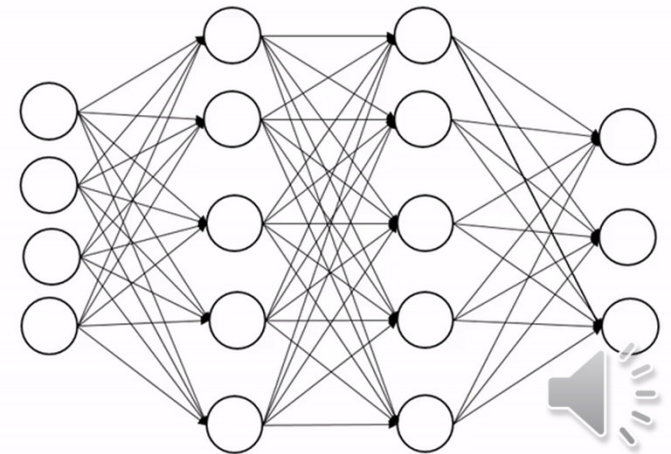
During training, a dropout layer randomly sets some of its input units to 0

- i.e., temporarily removing/dropping the node(s) from the layer

Simulating having multiple neural networks of different architectures to train the model.

Dropout layer is only applicable to the input and hidden layers

- i.e. cannot drop nodes for the output layer



Code Optimization

Traditional code optimization is still an important way to achieve acceptable performance.

After you've ensured that you're using as simple a model as you can

- identified which parts of your code are taking the most time
- then look at what you can do to speed them up

TensorFlow Lite for Microcontrollers code

- is written to run well across a large number of models and systems with as small a binary footprint as possible
- may still have optimizations that apply only to your particular model or platform
 - which you can benefit from by doing it yourself

Optimizing Operations

Default implementations of all operations in TensorFlow Lite for Microcontrollers

- are written to be small, understandable, and portable
 - NOT fast

So an approach that uses more lines of code or memory

- expected to increase the speed of operation

Examples:

- CMSIS-NN library
- TFLC has faster implementations version:
[the *kernels/portable optimized* directory](#)

Optimizing Operations

If you've not been able to find an optimized implementation of certain operations

- you might want to write your own
- focus only on making these operations faster rather than the general case

Examples:

- Trying a few different input and output sizes and parameters
- write some optimized code that exploited that pattern

A good first step is to find the innermost loop of the implementation.

- the section of code that will be run most frequently
- making improvements to it will have a bigger impact than for other parts of the algorithm

Taking Advantage of Hardware Features

One advantage of working with embedded systems

- they tend to be simpler in behaviour than more complex processors
- without deep instruction pipelines or caches

So it's a lot more feasible to identify potential performance using assembly-level optimizations

- without too much risk of unexpected side effects

More embedded systems that offer specialized hardware are also available

- to speed them up or reduce the power they need
- coprocessor-like accelerators that are able to respond with low latency
 - need to understand its operation to optimize its operation

Optimizing Energy Usage

After tackling the latency issues

- you can get your application working on a real device to test that it works the way you intend it to

The next priority is likely to be ensuring that your device has the lifetime it needs to be useful

- optimizing energy usage

The most important advantage that embedded devices have over desktop or mobile system

- they consume very little energy

Optimizing Energy Usage

The most challenging constraint for TinyML product

- battery life
- battery capacity is typically limited by the physical size of your product

Requiring human intervention to change or recharge batteries

- often not feasible

Useful lifetime of your device

- how long it will continue working will be determined by
 - how much energy it can store
 - how much energy it uses

Main area you can control to influence the lifetime of your device

- how much energy your system uses

Examples of power usage

Examples (From Chapter 16 of TinyML book):

- An Arm Cortex-A9 CPU can use between 500 and 2000 mW
- An Ambiq Cortex-M4F microcontroller might use 1 mW at 48 MHz clock
- A display might use 400 mW
- Active cell radio might use 800 mW
- Bluetooth might use 100 mW
- Bluetooth Low Energy might use 40 mW
- A microphone sensor might use 300 microwatts (μW)
- A 320×320 -pixel monochrome image sensor (like the Himax HM01B0) could use 1 mW at 30 FPS
- An accelerometer might use 1 mW

One top-level summary

- radio uses a lot more power than other functionality in an embedded product

Measuring Power Usage

Datasheets and estimates are helpful for planning

- but there's always something that doesn't fit into a simple model
- test the actual power usage

Most reliable method is to place a battery with a known capacity in the device and then see how long it lasts

- you might be aiming for a lifetime of months or years
 - most likely your first attempts will run for only hours or days
- it also captures all the effects you care about
 - such as failures when the voltage drops too low
 - which probably won't show up in simple modelling calculations

Estimating Energy Usage for a Model

Simplest way to estimate how much energy a model will use on a particular device

- measure the time for running one inference
- multiply the average power usage of the system with the time period to obtain the energy usage.

Can roughly estimate the time a model will take to execute

- by finding out how many arithmetic operations a model requires
 - and roughly how many operations per second a processor can perform

Datasheets will usually provide the power usage of a device at a particular frequency and voltage

- although it won't include common parts of the whole system such as memory or peripherals.

Estimating Energy Usage and Duration

Example:

- a model that takes 60 million operations to execute one inference
- an Arm Cortex-M4 running at 48 MHz
 - which can perform **two** 8-bit multiply/adds per cycle using its DSP extensions
- Estimate maximum latency
 - $60,000,000 / 2 / 48,000,000 = 625 \text{ ms}$
- If the chip uses 2 mW
 - Energy usage per inference = 1.25 mJ
- A CR2032 coin battery might hold 2,500 J
 - Life time = $2500\text{J} / 1.25\text{mJ} \approx 23 \text{ days}$

Improving Energy Usage – Duty Cycling

Once you know the approximate lifetime of your system

- You will probably be looking at ways to improve it
 - software focus approaches

One of the simplest ways of reducing power: using *duty cycling*

- insert sleeps between inference calls
- so that the processor spends more time in a low-power mode

Almost all embedded processors have the ability to put themselves into a sleep mode

- in which they don't perform any computation and use very little power
- but are able to wake up either after an interval
 - or when a signal comes in from outside

Improving Energy Usage – DMA & Frequency

Many modern microcontrollers have direct memory access (DMA) capabilities

- allow modules such as ADC to continuously sample and store the results in memory without any involvement from the main processor

You might also be able to reduce the frequency at which the processor executes instructions

- running slower can dramatically reducing the power it uses

Duty cycling and frequency reduction

- trade computation speed for power usage
- if you can also reduce the latency of your software
 - i.e. can trade for a lower power budget without affecting the performance

Improving Energy Usage – Cascading Design

One of the big advantages of machine learning

- easier to scale up or down the amount of compute and storage resources required
- while the accuracy will usually degrade gracefully

Can use what is known as a *cascade of models*.

Sensor data can be fed into a very small model with minimal compute requirements

- which may not be particularly accurate
- model tuned to have a high likelihood of triggering when a particular condition is present
 - even if it also produces a lot of false positives.

Improving Energy Usage – Cascading Design (cont.)

If the result indicates that something interesting has just happened

- the same inputs is then forward to a more complex model
 - to produce a more accurate result.
- which can potentially be repeated for several more stages

The inaccurate but tiny model can fit into a very power-efficient embedded device

- and running it continuously won't drain much energy

When a potential event is spotted

- a more powerful system can be woken up and a larger model run
- and so on down the cascade

Because the more powerful systems are operating for only a small fraction of the time

- their power usage doesn't break the budget.

Cascading Design

Example: The always-on voice interfaces on mobile phones

- A DSP is constantly monitoring the microphone, with a model listening for a wake word such as “Alexa”, “Siri”, “Hey Google”
- main CPU can be left in a sleep mode

When the DSP thinks it might have heard the right phrase

- send a signal to wake up the main CPU

The CPU can then run a much larger and more accurate model

- to confirm whether it really was the right phrase
- and send the following speech to an even more powerful processor in the cloud if it was

Summary

Various optimizing techniques can be used to reduce the latency and improve the energy usage for embedded product running ML applications

It is generally a good idea to focus on latency optimizations before energy

- because we will often need to validate that the design works first

Radio is very power-hungry

- limit its usage to rare occasions and for short periods such as by using cascading design

An embedded product might be able to achieve its goals

- even with a model that is not accurate enough
 - if it is able to spot most of the true positives, while the false positives occur at a low enough rate
- using cascading design